Parallel.Project.Strucutre.Updated

Updated Project Structure (With Modifications Highlighted)

> 🔴 **NEW** = Completely new addition
> 🟡 **MODIFIED** = Changed from original plan

Tech Stack

Layer	Choice
🟡 Backend (Core)	Java Raw TCP Sockets
🟡 Backend (Web Services)	Spring Boot REST (3 features)
Frontend	React
🔴 Bridge	Java WebSocket Server (React ↔ TCP)
Database	CockroachDB

Architecture

🟡 **Hybrid: Socket Server + Spring Boot Web Services + Distributed DB**

The main application runs as a **Java TCP Socket Server** handling all core marketplace logic. A **WebSocket bridge** connects the React frontend to the socket server. Additionally, **3 features are exposed as REST web services** via Spring Boot for requirement x. CockroachDB is partitioned across nodes.

```
React
(Browser)

WebSocket ──► Bridge Server (Port 8081)
(Main App)
                  │
                  │ TCP Socket
                  ▼
              Socket Server (Port 9000)
              (Core Business Logic)
                  │
```

```
└──── HTTP REST ────► Spring Boot (Port 8080)
      (3 Features)    (Web Services)
                          │
                          ▼
                     CockroachDB
                  (Distributed DB)
```

Partitioning strategy:

- Users and Items — horizontal partitioning by user ID range or region
- Transactions — partitioned by date range

Team Structure

Subteam	Size	Owns
🟡 Backend - Socket	3	TCP socket server, protocol handlers, all core features
🟡 Backend - Web Services	2	Spring Boot REST (3 features), 🔴 Partner Store API
Frontend	2	UI, user flows, screens
Database	1	Schema, distributed design, partitioning
🟡 DevOps/Integration	2	🔴 Bridge server, GitHub, coordination, report, diagrams

Sub-team Roles & Objectives

Backend - Socket 🟡

- 🔴 Design the application-level protocol (socket-protocol.md) before writing any code
- Design the API contract for web services (api-contract.md) before writing any code
- Implement all 9 core features over TCP sockets (auth, items, purchase, search, reports, etc.)
- Handle atomic transaction integrity for purchases (money + item transfer)
- 🔴 Handle multi-threaded client connections (one thread per client)
- Integrate with the DB layer via JDBC

Backend - Web Services 🟡

- Implement 3 features as REST web services via Spring Boot (requirement x)
- 🔴 Implement Partner Store API — interface allowing external stores to allow other stores to sell your products

- Share the same CockroachDB instance as the socket server

Frontend

- Build all UI screens for user-facing features
- Start on layout and screens using mock data before backend is ready
- 🟡 Connect to socket server via WebSocket (through bridge) for core features
- 🟡 Connect to Spring Boot via HTTP REST for the 3 web service features
- Own end-to-end user flow coherence

Database

- Design the full schema (Users, Items, Transactions, Inventory, etc.)
- Decide and implement the partitioning strategy
- Deliver `schema.sql`, `partitioning.sql`, and `seed.sql`
- Document justification for every partitioning decision

DevOps/Integration 🟡

- Manage GitHub — branching strategy, PRs, protect main and dev
- Own the report (.docx) and presentation (.pptx)
- Build and maintain the Gantt chart
- 🔴 Build and maintain the WebSocket-to-TCP bridge server
- Wire frontend to backend as features complete and catch integration breakage early
- Own demo session preparation

🔴 Application-Level Protocol (NEW SECTION)

Since the core communication uses raw TCP sockets, a custom protocol is defined:

```
Message Format:
  COMMAND|{JSON payload}

Response Format:
  STATUS|{JSON payload}

STATUS codes: OK, ERROR
```

Command Reference:

Command	Direction	Description
`REGISTER`	Client → Server	Create new account

Command	Direction	Description
`LOGIN`	Client → Server	Login to account
`LOGOUT`	Client → Server	Logout
`ADD_ITEM`	Client → Server	Add item for sale
`EDIT_ITEM`	Client → Server	Edit existing item
`REMOVE_ITEM`	Client → Server	Remove item
`SEARCH`	Client → Server	Search items by name/brand
`PURCHASE`	Client → Server	Buy an item
`DEPOSIT`	Client → Server	Add cash to account
`GET_ACCOUNT`	Client → Server	View account info
`GET_INVENTORY`	Client → Server	View/manage inventory
`GET_REPORT`	Client → Server	Generate reports
`OK`	Server → Client	Success response
`ERROR`	Server → Client	Error response with code

Examples:

```
→ REGISTER|{"username":"ahmed","email":"a@b.com","password":"hash123"}
← OK|{"userId":"uuid-123","message":"Account created"}

→ LOGIN|{"username":"ahmed","password":"hash123"}
← OK|{"token":"jwt-xxx","userId":"uuid-123"}

→ ADD_ITEM|{"token":"jwt-
xxx","name":"Laptop","brand":"Dell","price":15000,"quantity":1}
← OK|{"itemId":"uuid-456","message":"Item added"}

→ SEARCH|{"query":"laptop","brand":"dell"}
← OK|{"items":[{"itemId":"uuid-456","name":"Laptop","price":15000}]}

→ PURCHASE|{"token":"jwt-xxx","itemId":"uuid-456"}
← OK|{"transactionId":"uuid-789","message":"Purchase complete"}

← ERROR|{"code":401,"message":"Unauthorized"}
← ERROR|{"code":400,"message":"Insufficient funds"}
```

Full protocol documentation lives in `docs/socket-protocol.md`.

GitHub Branching Strategy

```
main
└── dev
    ├── feature/socket-server        🔴
    ├── feature/bridge               🔴
    ├── feature/protocol             🔴
    ├── feature/auth
    ├── feature/items
    ├── feature/purchase
    ├── feature/search
    ├── feature/reports
    ├── feature/inventory
    ├── feature/account
    ├── feature/webservices
    ├── feature/partner-api          🔴
    └── feature/frontend-[screen-name]
```

Rules:

- main — production-ready only, DevOps merges here at milestones
- dev — integration branch, all features merge here first
- feature/* — one branch per feature
- No direct pushes to main or dev — pull requests only
- PR to dev requires at least 1 reviewer approval

PR Flow:

```
feature/auth → (PR + review) → dev → (tested + stable) → main
```

Directory Structure

```
marketplace/
│
├── socket-server/                          🔴 NEW: Core backend
│   ├── src/main/java/com/marketplace/
│   │   ├── server/
│   │   │   ├── SocketServer.java           ← TCP server entry point
│   │   │   ├── ClientHandler.java          ← Thread per client
│   │   │   └── ProtocolParser.java         ← Parse COMMAND|JSON
│   │   ├── auth/
│   │   │   ├── AuthHandler.java
│   │   │   └── AuthService.java
│   │   ├── items/
│   │   │   ├── ItemHandler.java
│   │   │   └── ItemService.java
```

```
│   │   │   ├── purchase/
│   │   │   │   ├── PurchaseHandler.java
│   │   │   │   └── PurchaseService.java
│   │   │   ├── search/
│   │   │   │   ├── SearchHandler.java
│   │   │   │   └── SearchService.java
│   │   │   ├── inventory/
│   │   │   │   ├── InventoryHandler.java
│   │   │   │   └── InventoryService.java
│   │   │   ├── reports/
│   │   │   │   ├── ReportHandler.java
│   │   │   │   └── ReportService.java
│   │   │   ├── account/
│   │   │   │   ├── AccountHandler.java
│   │   │   │   └── AccountService.java
│   │   │   └── shared/
│   │   │       ├── models/
│   │   │       │   ├── User.java
│   │   │       │   ├── Item.java
│   │   │       │   ├── Transaction.java
│   │   │       │   └── Inventory.java
│   │   │       ├── db/
│   │   │       │   └── DatabaseManager.java    ← JDBC connection pool
│   │   │       ├── protocol/
│   │   │       │   ├── Request.java
│   │   │       │   └── Response.java
│   │   │       └── exceptions/
│   │   └── pom.xml
│
│   ├── bridge/                                  🔴 NEW: WebSocket ↔ TCP
│   │   ├── src/main/java/com/marketplace/
│   │   │   └── BridgeServer.java               ← WebSocket to TCP forwarder
│   │   └── pom.xml
│
│   ├── web-services/                            🟡 RENAMED (was backend/)
│   │   ├── src/main/java/com/marketplace/
│   │   │   ├── rest/
│   │   │   │   ├── Feature1Controller.java     ← REST web service 1
│   │   │   │   ├── Feature2Controller.java     ← REST web service 2
│   │   │   │   └── Feature3Controller.java     ← REST web service 3
│   │   │   ├── partner/                         🔴 NEW
│   │   │   │   ├── PartnerController.java      ← External store API
│   │   │   │   ├── PartnerService.java
│   │   │   │   └── PartnerRepository.java
│   │   │   ├── config/
│   │   │   │   └── WebServiceConfig.java
│   │   │   └── shared/
│   │   │       └── models/
│   │   ├── src/main/resources/
│   │   │   └── application.properties
```

```
│   └── pom.xml
│
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   │   ├── socketService.js          🔴 NEW: WebSocket client
│   │   │   └── apiService.js             🟡 REST client (3 features
only)
│   │   └── assets/
│   └── package.json
│
├── database/
│   ├── schema.sql
│   ├── partitioning.sql
│   └── seed.sql
│
├── docs/
│   ├── report.docx
│   ├── presentation.pptx
│   ├── api-contract.md
│   ├── socket-protocol.md                🔴 NEW: Protocol documentation
│   └── diagrams/
│       ├── architecture.png
│       ├── user-flow.png
│       └── db-schema.png
│
├── README.txt
└── .gitignore
```

🔴 Partner Store API (NEW SECTION)

Interface allowing external stores to allow other stores to sell your products

---

Development Model

Lightweight Agile — 3 sprints after Phase 1, with a Discord check-in every 2 days. DevOps/Integration subteam acts as de facto Scrum Master.

Sprint	Dates	Focus
🟡 1	Apr 25 – May 1	🔴 Socket Server + Bridge + Auth + Account + Items
🟡 2	May 2 – May 8	Search + Purchase + Deposit + Inventory + 🔴 Partner API

Sprint	Dates	Focus
🟡 3	May 9 – May 16	Reports + Web Services + Integration + Testing

---

Action Plan

Phase 1 — Design (Now → Apr 25)

Step 1 — Team Setup

- [x] ~~Create GitHub repo, add all members~~
- [x] ~~Create Discord channels: `#general`, `#backend`, `#frontend`, `#database`, `#devops`~~
- [x] ~~Create Miro board~~
- [x] ~~Assign sub-teams and leads~~
- [x] ~~Share and read requirements document together~~

Step 2 — Architecture Decision

- [x] ~~Agree on final tech stack~~
- [x] ~~🟡 Agree on architecture: Socket Server + Spring Boot Web Services + Distributed DB~~
- [x] ~~Draw system block diagram on Miro~~
  - [x] ~~Wait on backend~~
  - [x] ~~Wait on DB~~
- [x] ~~Draw user flow for each feature (i–ix) on Miro~~
- [ ] Redraw System Architecture Design

Step 3 — 🔴 Protocol Design (NEW)

- [ ] 🔴 Define all socket commands and their payloads
- [ ] 🔴 Write socket-protocol.md
- [ ] 🔴 Review protocol as a team before any coding starts

Step 4 — DB Design

- [x] ~~List all entities (User, Item, Transaction, Inventory, etc.)~~
- [x] ~~Draw ER diagram~~
- [x] ~~Write initial schema.sql~~
- [x] ~~Decide partitioning strategy per table and document justification~~

Step 5 — Phase 1 Document

- [x] ~~Create report.docx with cover page and table of contents~~
- [ ] Write Introduction section

- [ ] Write System Architecture section (include block diagram)
  - [ ] Draw user flow for each feature (i–ix) on Miro
  - [ ] Don't forget the 3rd party interface
    - [x] ~~Add high level architecture~~
      - [x] ~~Include documentation~~
  - [x] ~~Add Backend layout~~
- [ ] 🔴 Write Application Level Protocol section
- [ ] Write Distributed DB Design section (include ER + partitioning)
- [x] ~~Write initial Time Plan + Gantt chart~~

Step 6 — Submit Phase 1

- [ ] Review document as a group
- [ ] Submit by Apr 25

Between Phase 1 and 2 — Build (Apr 25 → May 16)

Step 7 — Project Bootstrap

- [ ] 🟡 Initialize socket-server project, push to dev
- [ ] 🔴 Initialize bridge project, push to dev
- [ ] 🟡 Initialize web-services (Spring Boot) project, push to dev
- [ ] Initialize frontend project, push to dev
- [ ] Set up CockroachDB locally for all backend developers
- [ ] 🟡 Finalize socket-protocol.md and api-contract.md before coding starts

Step 8 — DB Team

- [ ] Finalize schema.sql based on Phase 1 feedback
- [ ] Write partitioning.sql
- [ ] Write seed.sql with sample data for testing
- [ ] Set up a shared DB instance everyone can connect to

Step 9 — Backend Socket Server (in this order) 🟡

- [ ] 🔴 Socket server skeleton — SocketServer.java, ClientHandler.java, ProtocolParser.java
- [ ] Auth — register, login (JWT token generation)
- [ ] Account — view balance, purchased/sold items
- [ ] Items — add, edit, remove, list
- [ ] Deposit — add cash to account
- [ ] Search — text search by name and brand
- [ ] Purchase — atomic money + item transfer

- [ ] Inventory — manage stock
- [ ] Reports — transaction reports

Step 10 — Backend Web Services 🟡

- [ ] Pick 3 features to expose as REST endpoints via Spring Boot
- [ ] Implement the 3 REST web service endpoints
- [ ] 🔴 Implement Partner Store API

Step 11 — Bridge 🔴

- [ ] 🔴 Build WebSocket-to-TCP bridge server
- [ ] 🔴 Test that React can communicate through bridge to socket server

Step 12 — Frontend (parallel with backend)

- [ ] Build all screens using mock data first
- [ ] Screens: Register, Login, Dashboard, Item Listing, Search, Purchase, Deposit, Inventory, Reports
- [ ] 🔴 Implement socketService.js (WebSocket client for core features)
- [ ] 🟡 Implement apiService.js (HTTP client for 3 web service features only)
- [ ] Replace mock data with real connections as backend features complete

Step 13 — Integration

- [ ] Wire frontend to socket server (via bridge) feature by feature
- [ ] Wire frontend to Spring Boot for 3 web service features
- [ ] Fix any contract mismatches between frontend and backend
- [ ] Test full user flows end to end

Step 14 — Testing

- [ ] Test all socket commands manually
- [ ] Test all REST API endpoints (use Postman)
- [ ] Test all frontend screens
- [ ] Document all test cases and results for the report

Phase 2 — Documentation & Demo (→ May 16)

Step 15 — Final Report

- [ ] Update all sections from Phase 1 with final decisions
- [ ] 🔴 Write Application Level Protocol section (with full command reference)
- [ ] Write Testing section with all test cases and results

- [ ] Write End-User Guide with screenshots
- [ ] Write Role of Each Member section
- [ ] Write Resources section
- [ ] Add References

Step 16 — Presentation

- [ ] Build presentation.pptx covering all project phases
- [ ] Prepare live demo script
- [ ] Rehearse demo as a group

Step 17 — Final Submission

- [ ] Zip: source code, schema.sql, executables, resource files
- [ ] Include README.txt with compile and run instructions
- [ ] Submit on LMS by May 16

Summary of All Changes from Original Plan

#	What Changed	Why
🔴	Added Socket Server as core backend	TA requires low-level socket communication
🔴	Added WebSocket-TCP Bridge	React can't open raw TCP sockets, bridge needed
🔴	Added Application-Level Protocol	Required for socket communication + document section vii
🔴	Added Partner Store API	Project requires "interface to allow other stores to sell your products"
🟡	Spring Boot demoted to web services only	Only handles 3 REST features (requirement x), not main app
🟡	Backend team split into Socket + Web Services	Different tech, different responsibilities
🟡	Frontend connects via 2 channels	WebSocket for core features, HTTP REST for 3 web services
🟡	Sprint 1 now includes socket/bridge bootstrap	Must be working before any feature development
🟡	Directory structure reorganized	3 backend components instead of 1

#	What Changed	Why
🔴	New Protocol Design step in Phase 1	Must be defined before coding starts